
CS 301

High-Performance Computing

Assignment 03 - Serial Interpolation on Scattered Data

Diya Patel (202301216)
Yesha Joshi (202301462)

February 18, 2026

Contents

1	Introduction	3
2	Problem Description	3
3	Interpolation Method	3
3.1	Cell Identification	3
3.2	Local Coordinates	3
3.3	Bilinear Weights	3
4	Implementation Approach	4
4.1	Data Structures	4
4.2	Optimized Interpolation Strategy	4
4.3	Algorithm Workflow	4
5	Time Complexity Analysis	5
6	Arithmetic Operations Per Particle	5
7	Memory Access Analysis	5
8	Arithmetic Intensity	5
9	Performance Results	6
9.1	Execution Time Table	6
9.2	Execution Time Plot	6
10	Discussion	6
11	Conclusion	7

1 Introduction

This assignment focuses on implementing and optimizing a serial bilinear interpolation algorithm for mapping scattered points onto a structured grid.

Given scattered particles (x_i, y_i) in a unit domain, the objective is to distribute their values onto a structured mesh of size $NX \times NY$ using bilinear interpolation.

The implementation emphasizes correctness, memory layout, floating-point efficiency, and performance analysis.

2 Problem Description

Given a set of scattered points:

$$S = \{(x_i, y_i, f_i)\}$$

where $f_i = 1$ for all particles, the task is to interpolate these values onto a structured mesh grid:

$$G = \{(X_i, Y_j)\}$$

where:

$$\Delta x = \frac{1}{NX}, \quad \Delta y = \frac{1}{NY}$$

Each particle contributes to the four surrounding grid nodes.

3 Interpolation Method

For a particle located at (x, y) :

3.1 Cell Identification

$$i = \left\lfloor \frac{x}{\Delta x} \right\rfloor, \quad j = \left\lfloor \frac{y}{\Delta y} \right\rfloor$$

3.2 Local Coordinates

$$l_x = x - X_i, \quad l_y = y - Y_j$$

3.3 Bilinear Weights

$$w_{i,j} = \frac{(\Delta x - l_x)(\Delta y - l_y)}{\Delta x \Delta y}$$

$$w_{i+1,j} = \frac{l_x(\Delta y - l_y)}{\Delta x \Delta y}$$

$$w_{i,j+1} = \frac{(\Delta x - l_x)l_y}{\Delta x \Delta y}$$

$$w_{i+1,j+1} = \frac{l_x l_y}{\Delta x \Delta y}$$

These weights ensure:

$$w_{i,j} + w_{i+1,j} + w_{i,j+1} + w_{i+1,j+1} = 1$$

4 Implementation Approach

4.1 Data Structures

- 1D array for mesh storage (row-major layout)
- Structure array for storing particle coordinates

Mesh indexing:

$$\text{index} = j \times \text{GRID_X} + i$$

4.2 Optimized Interpolation Strategy

- Precompute $1/\Delta x$, $1/\Delta y$
- Avoid divisions inside particle loop
- Reuse computed quantities
- Minimize index recalculation

4.3 Algorithm Workflow

1. Read input file
2. Initialize structured mesh to zero
3. Loop over all particles
4. Compute indices and weights
5. Accumulate contributions
6. Repeat for *Maxiter* iterations
7. Output mesh

5 Time Complexity Analysis

The main interpolation loop iterates over all particles.

$$T(N) = \mathcal{O}(N)$$

Since interpolation runs for *Maxiter* iterations:

$$T_{total} = \mathcal{O}(N \times \text{Maxiter})$$

Grid size does not affect loop length directly.

6 Arithmetic Operations Per Particle

Approximately 24 floating-point operations per particle:

- 2 multiplications: $x \cdot inv_dx, y \cdot inv_dy$
- 2 integer casts
- 2 boundary comparisons
- 2 multiplications: $i \cdot dx, j \cdot dy$
- 2 subtractions: l_x, l_y
- 2 subtractions: $(dx - l_x), (dy - l_y)$
- 4 multiplications: weight computation
- 4 multiplications: multiply by inv_area
- 4 additions: mesh accumulation

7 Memory Access Analysis

Per particle:

- 2 memory reads (x, y)
- 4 memory writes (grid updates)

Memory access is irregular due to particle distribution, leading to cache sensitivity.

8 Arithmetic Intensity

$$\text{Arithmetic Intensity} = \frac{24 \text{ FLOPs}}{6 \text{ memory ops}} \approx 3.6$$

The algorithm is moderately memory-bound.

9 Performance Results

9.1 Execution Time Table

Case	Lab PC (s)	Cluster (s)	Speedup (Lab/Cluster)
(a)	0.084440	0.260000	0.325
(b)	0.470347	1.150000	0.409
(c)	0.347945	0.930000	0.374
(d)	1.896351	4.720000	0.402
(e)	1.874231	3.280000	0.571

Table 1: Execution time comparison between Lab PC and HPC Cluster

9.2 Execution Time Plot

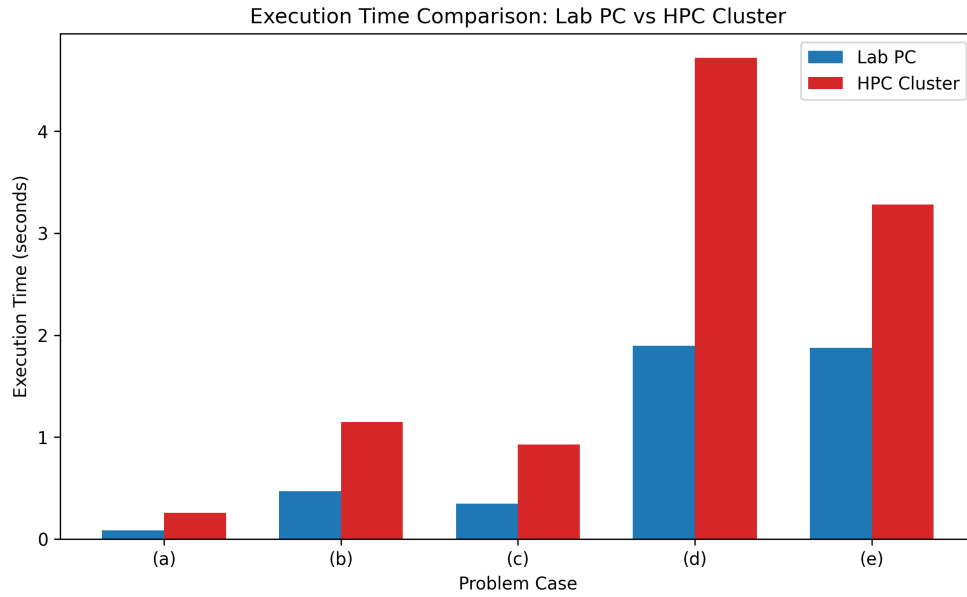


Figure 1: Execution time comparison between Lab PC and HPC Cluster

10 Discussion

From the results, the following observations can be made:

- For every problem configuration (a)–(e), the Lab PC demonstrates lower execution time compared to the HPC Cluster.
- Execution time increases with the number of particles, confirming the linear time complexity of the interpolation algorithm.

- The performance gap between the two systems becomes more pronounced for larger problem sizes (cases d and e). The widening gap for larger cases suggests that memory access behavior plays a significant role in overall performance.
- The interpolation algorithm performs approximately six memory operations per particle (two reads and four writes). Since mesh updates occur at positions dependent on particle location, the memory access pattern is irregular.
- As the number of particles increases, the number of scattered memory writes also increases proportionally. This leads to greater pressure on the memory subsystem and cache hierarchy.

These results indicate that, for the current serial implementation, the Lab PC achieves better single-thread performance compared to the HPC Cluster.

Overall, the experimental results validate the theoretical time complexity analysis and demonstrate predictable performance growth across all cases.

11 Conclusion

The serial interpolation algorithm was implemented and optimized with careful attention to memory layout and arithmetic efficiency.

The algorithm demonstrates linear scaling with respect to the number of particles and exhibits memory-bound characteristics for large problem sizes.